

Lab 2 Report

Part I

In this part, I have created a program to simulate Packet Generalized Processor Sharing (PGPS). It is packet based GPS algorithm used in fair queuing for computer networks. The main goal of this part is to ensure by simulating, that packets from different flows are scheduled in a fair way proportional to their assigned weights. The entire program has utilized OOP (with java) to structure and represent flows, events, packets, time to classes and objects. Additionally, an argument *-verbose* can be passed during the program launch to log to console as well. It is just something that I liked to implement to have some “feature” in the simulation.

The simulation starts by reading flow information (each flow’s weight) and packet arrival data from an input file. Each packet is assigned a virtual start time and virtual finish time based on the system’s virtual clock. These times reflect when the packet would be served in the ideal GPS system. Flows also keep track of their last virtual finish time too, to ensure packets from the same flow are served in order. The *Flow*, *Clock*, *Packet*, and *Event* are classified separately as classes with proper members and/or helper methods, just to mimic the idea/process into code. After all, OOP in general is very good at providing such structural way of managing things.

In the simulation, two priority queues are used. i.e. *eligibleDepartures*, and *futureDepartures*. When a packet arrives, the simulation updates the virtual time based on the total weight of currently backlogged flows. The packet’s virtual start and finish times are calculated and it is placed into the appropriate queue. And if the system is idle and there are eligible packets, the packet with the smallest virtual finish time starts being processed. Methods like *getTotalBackloggedWeight*, *changeClockTime* helps in carefully advancing time either to the next arrival or the finish of the current packet, whichever comes first of course. When a packet finishes, the virtual clock is set exactly to its finish time, the flow’s backlog count is updated, and any newly eligible packets are advanced further. Doing so, the program mimics the behavior of an ideal fair queuing system but with practical packet based scheduling.

Part II

This part implements Worst-case Fair Weighted Fair Queuing (WF²Q), which is a more accurate and fair packet scheduling algorithm than PGPS. It guarantees that every packet is served as close as possible to the ideal GPS model. This way, it ensures strong fairness and very minimal delay deviation between actual and ideal process. As we already know, the change is made on top of the code from part I (PGPS). In the PGPS version, as soon as a packet arrives, it could be scheduled for departure right away based on purely its virtual finish time. But with WF²Q, we improve the approach by adding an extra “eligibility” check. To sum it up, packets are first placed into *futureDepartures* (as its name suggest) holding the area if they arrive too early, and it moves to the *eligibleDepartures* queue and become a proper candidate for processing only when the system’s virtual time reaches the packet’s virtual start time. And because of this extra step, the algorithm never serves a packet earlier than it should.